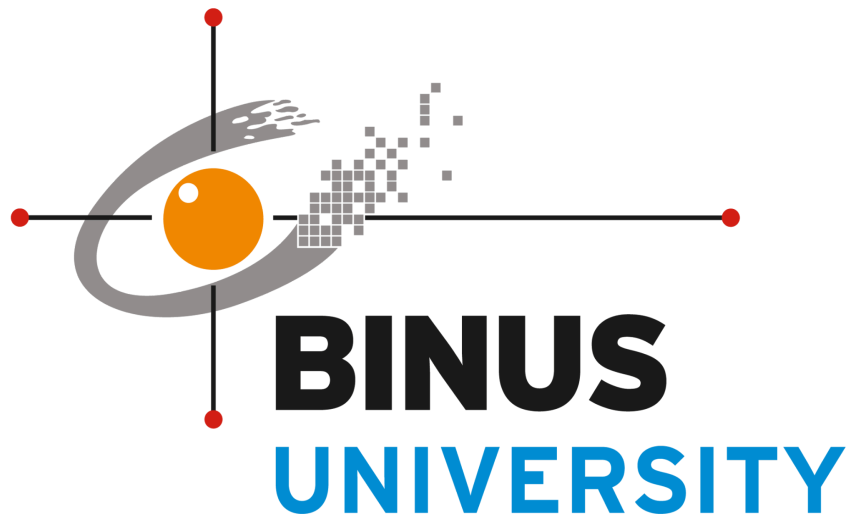


FINAL REPORT
PROJECT BASED LEARNING



**Klasifikasi Citra Bencana Alam (Earthquake, Urban Fire, Landslide,
Water Disaster) Menggunakan Algoritma CNN**

Oleh:

Kelas: LC09

2802446560 – Muhammad Akhdan Athallah

2802444901 – Muhammad Hylmi Razzan

Course:

DTSC6007001 – Deep Learning

DATA SCIENCE
SCHOOL OF COMPUTER SCIENCE
BINA NUSANTARA UNIVERSITY

I. Introduction

Bencana alam seperti gempa bumi (*earthquake*), kebakaran perkotaan (*urban fire*), tanah longsor (*landslide*), dan bencana air (*water disaster*) sering terjadi dan membutuhkan penanganan yang sangat cepat. Saat sebuah bencana terjadi, ada banyak sekali foto visual yang tersebar, baik dari masyarakat maupun petugas di lapangan. Sayangnya, memilah dan mengenali jenis bencana dari banyaknya foto tersebut secara manual membutuhkan waktu yang lama, padahal tim penyelamat membutuhkan informasi yang cepat untuk mengambil tindakan. Untuk mengatasi kendala waktu tersebut, kerangka kerja global dari PBB secara spesifik mendorong pengembangan inovasi dan teknologi sebagai solusi terarah untuk menyelesaikan berbagai hambatan dalam manajemen risiko bencana (UNDRR, 2015).

Untuk mengatasi tantangan tersebut, proyek ini bertujuan membangun sebuah sistem klasifikasi gambar otomatis menggunakan algoritma Convolutional Neural Network (CNN) dan Multilayer Perceptron (MLP). Algoritma CNN dipilih karena kemampuannya yang sangat baik dalam mengenali dan mengekstrak pola visual penting pada sebuah foto. Setelah CNN menyaring fitur-fitur dari gambar tersebut, tahap klasifikasi kemudian dilakukan oleh algoritma MLP untuk menentukan jenis bencananya. Dengan melatih model ini menggunakan kumpulan data gambar dari empat jenis kejadian, yaitu gempa bumi, kebakaran perkotaan, tanah longsor, dan bencana air, sistem diharapkan mampu mengidentifikasi jenis bencana secara cepat dan otomatis. Solusi teknologi ini pada akhirnya bertujuan untuk membantu pihak berwenang dalam mempercepat respons darurat dan pengambilan keputusan saat krisis terjadi.

Dalam proyek ini, kumpulan data gambar yang kami gunakan memiliki masalah ketidakseimbangan jumlah yang cukup parah. Berikut adalah rinciannya:

Kelas Data	Jumlah Gambar
Water Disaster	1035
Landslide	456
Urban Fire	419
Earthquake	36

Perbedaan jumlah yang sangat jauh antara kategori Bencana Air dan Gempa Bumi ini akan menjadi masalah saat proses pelatihan algoritma. Sistem kecerdasan buatan belajar dengan cara mengenali dan menghafal pola dari gambar yang dimasukkan ke dalamnya.

Karena sistem memproses gambar “Bencana Air” (*water disaster*) ribuan kali, sistem akan sangat memahami pola visual dari bencana tersebut. Sebaliknya, karena gambar “Gempa Bumi” (*earthquake*) sangat sedikit, sistem kekurangan informasi untuk bisa benar-benar mengenali pola gambar gempa bumi.

Akibatnya, sistem klasifikasi akan menjadi bias atau berat sebelah. Jika data langsung digunakan, sistem akan cenderung menebak gambar apa pun sebagai "Bencana Air" (*water disaster*) karena itu adalah label kelas yang paling dominan, dan sistem akan sering gagal saat diminta mengidentifikasi gambar "Gempa Bumi" (*earthquake*). Oleh karena itu, kondisi ketimpangan data ini menjadi tantangan teknis yang penting untuk diselesaikan agar sistem bisa mengenali keempat jenis bencana dengan baik.

II. Methods

Bagian ini menyajikan kerangka kerja yang diusulkan untuk klasifikasi bencana alam multi-kelas, meliputi Gempa Bumi (*earthquake*), Kebakaran Perkotaan (*Urban Fire*), Tanah Longsor (*Landslide*), dan Bencana Air (*Water Disaster*). Metodologi ini menggabungkan ekstraksi fitur *deep convolutional* melalui model CNN dari awal (*Scratch Model*) dan *Transfer Learning* dengan mekanisme penanganan ketidakseimbangan data (*Class Weight* dan *Oversampling*) untuk menghasilkan prediksi yang akurat dan andal. Sistem yang diusulkan mengikuti arsitektur *pipeline* modular yang mencakup delapan komponen utama: konfigurasi, analisis eksploratif (EDA), *preprocessing* data, pemuatan dan pembagian dataset, augmentasi data, ekstraksi fitur mendalam, klasifikasi multi-kelas, serta evaluasi menyeluruh.

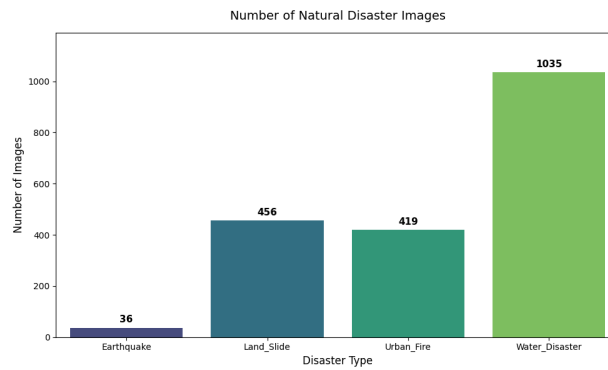


1. Configuration

Tahap persiapan awal ini sangat krusial agar seluruh pemrosesan data berjalan terukur. Kami mulai dengan menetapkan jalur (*path*) ke folder dataset di Google Drive dan mendefinisikan label kelas secara manual (Earthquake, Land_Slide, Urban_Fire, Water_Disaster) agar model nantinya hanya fokus pada kategori tersebut. Selanjutnya, kami mengatur parameter pemrosesan, yaitu ukuran gambar 224x224 piksel untuk penyeragaman, dan batch size sebesar 32 agar memori sistem tetap stabil. Terakhir, kami menetapkan nilai seed 42 untuk mengunci unsur acak, sehingga hasil pemrosesan dan pembagian data akan selalu konsisten saat kami melakukan evaluasi berulang kali.

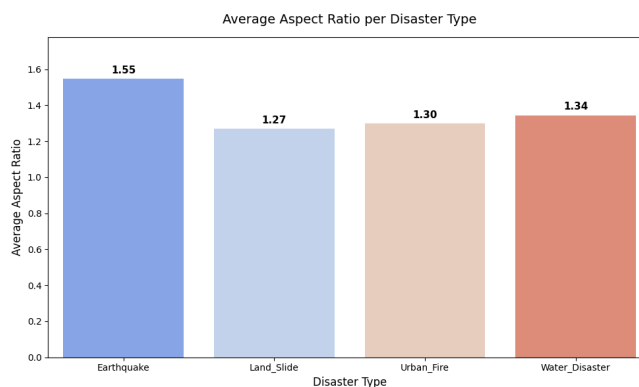
2. EDA

Setelah konfigurasi, kami melakukan Exploratory Data Analysis (EDA) dengan membaca file langsung dari Google Drive. Pertama, kami mengecek sebaran jumlah dan contoh gambar per kelas untuk mendeteksi potensi ketidakseimbangan data (*data imbalance*) yang bisa membuat model menjadi bias.



Hasil EDA menunjukkan adanya *data imbalance* yang cukup ekstrem pada persebaran kelas, paling terlihat pada kelas Earthquake. Untuk mengatasi masalah tersebut, kami akan menggunakan pendekatan *class weight* dan *oversampling* pada setiap model kami.

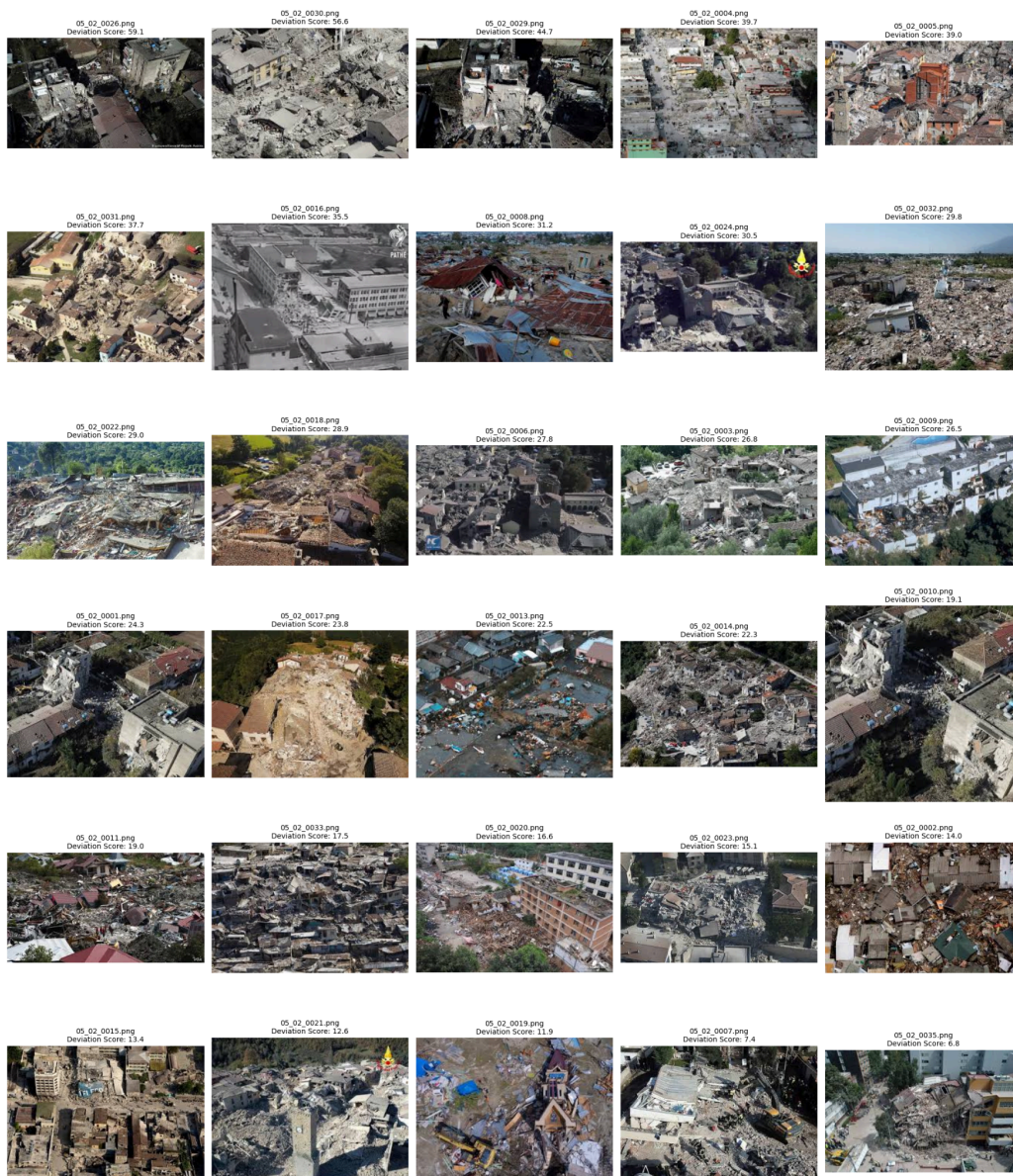
Kedua, kami menghitung rata-rata rasio panjang dan lebar gambar sebagai acuan untuk melakukan tindakan penyesuaian ukuran yang akan diambil, antara memotong bagian tepinya (*cropping*) atau menarik gambarnya secara paksa (*stretching*) agar bentuk asli objek tidak terlihat gepeng saat dimasukkan ke model.



Hasil EDA menunjukkan rata-rata rasio aspek gambar di setiap kelas relatif jauh dari 1.00, khususnya pada kelas Earthquake. Oleh karena itu, kami memilih metode *cropping* dibandingkan *stretching*. Pemaksaan *stretching* akan membuat gambar terdistorsi secara drastis, yang justru membuat model kesulitan dalam mengenali pola aslinya.

Ketiga, kami mengecek ekstremitas pencahayaan dengan menampilkan sampel foto paling gelap dan paling terang. Dan terakhir, kami menganalisis komponen warna RGB untuk mengidentifikasi 30 data pencilan (*outlier*) teratas yang coraknya paling melenceng. Daftar outlier ini menjadi patokan utama untuk pembuangan gambar secara manual di tahap berikutnya.

Earthquake Darkest Value: 67.84  Brightest Value: 135.11 	Landslide Darkest Value: 58.47  Brightest Value: 189.76 
Urban Fire Darkest Value: 17.19  Brightest Value: 181.40 	Water Disaster Darkest Value: 24.06  Brightest Value: 213.48 



Hasil EDA pada top 30 *outlier* nilai RGB menunjukkan adanya 19 file gambar yang tidak relevan di beberapa kelas. Kesembilan belas file tersebut telah kami hapus secara manual, yaitu :

'01_01_0339.png', '04_01_0441.png',
 '03_0786.png', '04_01_0451.png',
 '04_01_0443.png', '04_01_0107.png',
 '04_01_0392.png', '04_01_0008.png',
 '04_01_0047.png', '04_01_0102.png',
 '04_01_0345.png', '04_01_0220.png',
 '04_01_0104.png', '04_01_0069.png',
 '04_01_0138.png', '04_01_0218.png',
 '04_01_0348.png', '04_01_0040.png',
 '04_01_0276.png'



3. Preprocessing

Tahap ini dieksekusi langsung pada sumber dataset di dalam Google Drive, sehingga data yang dibuang akan terhapus permanen dari sumbernya. Proses pembersihan mencakup penghapusan data duplikat, file yang korup, dan gambar kosong. Perhatian khusus diberikan pada kelas Earthquake agar tidak ada pengurangan keliru, mengingat datanya sangat terbatas (hanya 36 gambar). Selain pembersihan otomatis tersebut, kami juga melakukan

penghapusan manual terhadap 30 data outlier teratas berdasarkan nilai RGB yang sudah ditemukan pada tahap EDA sebelumnya.

4. Load Data

Setelah data di Google Drive dipastikan sudah bersih melalui tahap preprocessing, barulah kami masuk ke tahap Load Data. Tahap ini berfungsi sebagai proses injeksi, di mana data gambar yang sudah rapi tersebut ditarik ke dalam memori Google Collab untuk diubah formatnya menjadi dataset TensorFlow. Proses pemuatan data ini memanggil kembali parameter yang sudah kami tetapkan pada tahap konfigurasi awal, yaitu ukuran gambar 224x224 piksel dan ukuran batch sebesar 32.

Satu hal yang sangat penting, saat proses injeksi ini berlangsung, kami juga sekaligus mengeksekusi solusi untuk masalah rasio gambar yang sudah kami analisa pada tahap EDA. Kami mengaktifkan fitur pemotongan rasio aspek (*crop to aspect ratio*) di dalam fungsi pemuatan data TensorFlow. Dengan pengaturan ini, sistem akan secara cerdas memotong bagian tepi gambar sambil mempertahankan proporsi asli benda di dalamnya. Hasilnya, saat gambar disesuaikan menjadi format kotak 224x224 piksel, objek bencana di dalam foto sama sekali tidak mengalami distorsi atau terlihat gepeng.

Dengan selesainya tahap Load Data ini, seluruh gambar yang sudah bersih dan proporsional beserta label kelasnya telah berhasil diintegrasikan ke dalam sistem memori Google Colab.

5. Split Data

Setelah dataset berhasil dimuat, langkah selanjutnya adalah memecah kumpulan gambar tersebut menjadi tiga kelompok sesuai fungsinya. Porsi terbesar yaitu 70 persen dialokasikan sebagai data latih (*training*) agar model AI bisa belajar mengenali pola bencana. Kemudian, 15 persen digunakan sebagai data validasi (*validation*) untuk memantau dan mengoreksi performa model saat proses belajar sedang berlangsung. Sisa 15 persen terakhir disimpan khusus sebagai data uji (*testing*) untuk bahan evaluasi akhir menggunakan gambar yang belum pernah dilihat oleh model sama sekali.

Karena data kami sudah dibungkus dalam bentuk batch, proses pemecahannya dilakukan menggunakan fungsi pengambilan dan pelewatan urutan dari TensorFlow. Sistem mengambil 70 persen batch pertama untuk data latih, lalu melewati bagian tersebut untuk mengambil 15 persen batch berikutnya sebagai data validasi, dan sisanya otomatis menjadi data uji. Kemudian, kami melakukan *reshuffle* pada data latih agar urutan sampel teracak ulang setiap epoch, sehingga model tidak mempelajari pola berdasarkan urutan data dan proses pelatihan menjadi lebih umum.

6. Augmentation

Sebelum masuk ke tahap pemodelan, data yang sudah di split tadi harus melewati satu tahap persiapan akhir yaitu augmentasi. Langkah pertama yang kami lakukan pada tahap ini adalah normalisasi gambar (*rescaling*). Sistem akan mengubah rentang nilai warna piksel

yang tadinya 0 sampai 255 menjadi angka desimal kecil antara 0 hingga 1. Penyesuaian angka ini sangat krusial agar beban komputasi menjadi lebih ringan dan model bisa belajar dengan jauh lebih cepat.

Setelah gambar dinormalisasi, kami menginjeksi variasi tambahan khusus untuk train melalui proses augmentasi. Gambar train tersebut akan dimodifikasi secara acak oleh sistem, seperti dibalik secara horizontal, diputar hingga 10 persen, serta diperbesar atau diperkecil (*zoom*) sebesar 10 persen. Variasi buatan ini bertujuan untuk memperkaya keragaman sudut pandang gambar, sehingga model kami tidak mudah menghafal data dan menjadi lebih *robust* saat dihadapkan pada gambar baru. Sangat penting untuk dicatat, proses augmentasi ini sama sekali tidak diterapkan pada data validation dan test agar keaslian data evaluasi tetap terjaga murni.

Seluruh proses normalisasi dan augmentasi di atas dijalankan menggunakan fitur paralelisasi (AUTOTUNE) dan prefetching dari TensorFlow. Fitur ini memungkinkan komputer untuk menyiapkan data berikutnya saat model sedang sibuk memproses data sebelumnya, sehingga waktu pelatihan menjadi jauh lebih efisien. Terakhir, kami melakukan pengecekan akhir (*sanity check*) dengan menampilkan satu gambar acak beserta labelnya, mengecek dimensi matriks data, dan melihat langsung nilai warna pikselnya. Pengecekan visual dan angka ini menjadi bukti bahwa gambar sudah dinormalisasi dengan benar dan dataset kami sudah siap untuk dimasukkan ke tahap Modelling.

7. Modelling

Mengingat distribusi label kelas pada dataset kami tidak seimbang (*data imbalance*), kami akan menangani masalah ini menggunakan dua pendekatan berbeda untuk masing-masing model (Scratch model, Modified model), yaitu melalui penerapan *class weight* dan teknik *oversampling*.

7.1 Scratch Model

Pada Scratch Model, kami membangun arsitektur CNN dari awal menggunakan empat hidden layer dengan jumlah neuron 32, 64, 128, dan 64. Seluruh hidden layer menggunakan aktivasi ReLU agar model mampu mempelajari pola visual yang kompleks pada gambar bencana.

Pada tiga layer awal, diterapkan Max Pooling untuk mengurangi dimensi fitur sekaligus mempertahankan informasi penting dari gambar. Setelah proses ekstraksi fitur selesai, data diratakan menggunakan Flatten sebelum masuk ke layer berikutnya.

Setelah hidden layer terakhir, digunakan Dropout sebesar 0.2 untuk mengurangi risiko overfitting dengan menonaktifkan sebagian neuron secara acak selama training. Pada bagian output, digunakan 4 neuron dengan aktivasi Softmax untuk menghasilkan probabilitas klasifikasi empat kategori bencana. Secara keseluruhan, model ini memiliki 5,631.364 parameter dan seluruh parameter dilatih secara penuh selama proses training.

Pada tahap training, model menggunakan optimizer Adam dengan learning rate 0.001, loss function sparse categorical crossentropy, dan metrik accuracy. Training dijalankan maksimal 50 epoch dengan bantuan callback ReduceLROnPlateau untuk menurunkan learning rate saat performa stagnan, serta EarlyStopping untuk menghentikan training lebih awal dan menyimpan weight terbaik model.

7.1.1 *Class Weight*

Pada pendekatan *class weight*, *data imbalance* ditangani tanpa menambah jumlah data asli. Bobot setiap kelas dihitung menggunakan scikit-learn berdasarkan distribusi data pada dataset.

Kelas minoritas seperti Earthquake diberikan *weight* yang lebih besar, sedangkan kelas mayoritas memperoleh *weight* yang lebih kecil. *Weight* ini diterapkan langsung saat proses training sehingga model memberikan perhatian lebih besar pada kelas dengan jumlah data sedikit. Dengan cara ini, proses pembelajaran menjadi lebih seimbang dan tidak terlalu bias terhadap kelas mayoritas.

7.1.2 *Oversampling*

Pada pendekatan *oversampling*, *data imbalance* diatasi dengan menyeimbangkan jumlah data pada setiap kelas. Teknik ini dilakukan dengan memperbanyak sampel pada kelas minoritas hingga jumlahnya setara dengan kelas mayoritas.

Setelah distribusi data menjadi seimbang, setiap kategori bencana memiliki proporsi data yang sama selama training. Hal ini membantu Scratch Model belajar lebih adil terhadap seluruh kelas dan mengurangi bias terhadap kategori tertentu. Namun, karena beberapa data minoritas digunakan berulang, pendekatan ini juga memiliki risiko *overfitting* pada kelas minoritas.

7.2 *Trained Model Under 10M Parameters*

Pada tahap ini, kami menerapkan transfer learning menggunakan arsitektur EfficientNetB1 yang telah dilatih menggunakan dataset ImageNet, sehingga model sudah mampu mengenali berbagai pola visual dasar dengan efisien dan jumlah parameter yang relatif kecil.

Sebelum masuk ke model, gambar diproses menggunakan fungsi *preprocess_input* library bawaan EfficientNet agar format input sesuai dengan standar model asli. Proses preprocessing ini dijalankan secara paralel menggunakan AUTOTUNE untuk meningkatkan efisiensi pipeline data.

Setelah model dasar EfficientNetB1 di-*load*, seluruh *weight* pada base model di-*freeze* agar parameter awal tidak berubah selama training. Selanjutnya, ditambahkan beberapa layer baru untuk klasifikasi empat kelas bencana.

Layer pertama adalah Global Average Pooling untuk merangkum fitur visual tanpa menambah terlalu banyak parameter. Setelah itu digunakan Batch Normalization agar proses training lebih stabil. Data kemudian diteruskan ke hidden layer berisi 128 neuron dengan aktivasi ReLU untuk mempelajari pola gambar yang lebih spesifik.

Untuk mengurangi risiko *overfitting*, diterapkan Dropout sebesar 0.2 yang akan menonaktifkan sebagian neuron secara acak selama training. Pada bagian akhir, digunakan output layer dengan 4 neuron dan aktivasi Softmax untuk menghasilkan probabilitas klasifikasi setiap kategori bencana.

Proses training menggunakan *optimizer* Adam dengan *learning rate* 0.001, metrik *accuracy*, dan *loss function* *sparse categorical crossentropy*. Training dijalankan hingga maksimal 50 epoch dengan bantuan ReduceLROnPlateau untuk menurunkan *learning rate* saat performa stagnan, serta EarlyStopping untuk menghentikan training lebih awal dan menyimpan *weight* terbaik model.

7.2.1 Class Weight

Untuk menangani *data imbalance* pada model EfficientNetB1, digunakan pendekatan *class weight* seperti pada Scratch Model. Berbeda dengan oversampling yang menambah distribusi data, metode ini langsung memberikan bobot khusus pada setiap kelas selama proses training menggunakan perhitungan dari scikit-learn.

Dengan cara ini, model akan memberikan perhatian lebih besar pada kelas minoritas, terutama Earthquake, sehingga proses pembelajaran menjadi lebih seimbang dan tidak terlalu bias terhadap kelas mayoritas. Pendekatan ini juga membuat proses transfer learning tetap efisien tanpa perlu mengubah jumlah data asli pada dataset training.

7.2.2 Oversampling

Pada tahap oversampling untuk model EfficientNetB1, *imbalance data* antar kelas diatasi dengan menyeimbangkan jumlah sampel pada *data training*. Dataset terlebih dahulu di-*unbatch* lalu dipisahkan berdasarkan masing-masing kategori bencana.

Setiap kelas kemudian dibuat *repeat* agar data tidak habis saat proses sampling. Setelah itu, sistem membentuk dataset baru menggunakan *balanced sampling* dengan proporsi yang sama untuk setiap kelas, sehingga kelas minoritas memiliki jumlah data yang setara dengan kelas mayoritas.

Karena dataset telah disusun ulang, data kemudian di-*shuffle* agar model tidak menghafal urutan gambar. Setelah itu, data dikembalikan ke dalam bentuk *batch* sebelum masuk ke pipeline training.

Tahap selanjutnya adalah *preprocessing* menggunakan *process_input* library milik EfficientNet dan penerapan *data augmentation* untuk meningkatkan variasi gambar serta membantu generalisasi model.

Sebagai tahap akhir, digunakan *prefetching* agar proses *training* berjalan lebih efisien. Seluruh proses *oversampling* dan *augmentation* hanya diterapkan pada *data training*, sedangkan *data validation* dan *testing* tetap menggunakan data asli tanpa manipulasi.

8. Evaluation

Tahap evaluasi dilakukan setelah proses *training* selesai untuk mengukur seberapa baik model dalam melakukan klasifikasi gambar bencana alam. Evaluasi dilakukan menggunakan *data testing* yang belum pernah dilihat model sebelumnya agar hasil pengukuran benar-benar merepresentasikan kemampuan generalisasi model terhadap data baru.

Pada penelitian ini, proses evaluasi tidak hanya berfokus pada *accuracy*, tetapi juga menggunakan Classification Report dan Confusion Matrix. Hal ini dikarenakan dataset memiliki kondisi *data imbalance*, sehingga *accuracy* saja belum cukup untuk menggambarkan performa model secara menyeluruh. Dengan kombinasi beberapa metode evaluasi tersebut, performa model dapat dianalisis lebih detail, baik dari sisi tingkat ketepatan prediksi, kemampuan mendeteksi setiap kelas, maupun pola kesalahan klasifikasi yang dihasilkan model.

8.1 Accuracy and Loss

Pada tahap evaluasi awal, kinerja model dipantau menggunakan dua metrik utama, yaitu *accuracy* dan *loss*. Metrik *accuracy* digunakan untuk mengukur seberapa tepat model dalam mengklasifikasikan gambar ke dalam empat kategori bencana alam secara keseluruhan. Sementara itu, metrik *loss* menggunakan *sparse categorical crossentropy* berfungsi untuk mengukur seberapa besar penyimpangan atau tingkat kesalahan antara prediksi yang dihasilkan oleh model dengan label aslinya.

Pemantauan kedua metrik ini dilakukan secara bersamaan pada *training data* dan *data validation data* di setiap *epoch*. Perbandingan pergerakan nilai *training* dan *validation* ini sangat krusial untuk mendeteksi perilaku belajar model melalui visualisasi kurva. Melalui kurva ini, dapat diidentifikasi apakah model berhasil belajar dengan baik (*good fit*), gagal menangkap pola fitur gambar (*underfitting*), atau justru terlalu menghafal data latih sehingga rentan gagal saat mengenali gambar baru (*overfitting*). Evaluasi pergerakan *accuracy* dan *loss* juga menjadi parameter utama untuk memvalidasi apakah mekanisme pengoptimalan seperti EarlyStopping dan ReduceLROnPlateau telah bekerja secara efektif dalam menjaga stabilitas proses pembelajaran model.

8.2 Classification Report

Penggunaan metrik *accuracy* secara keseluruhan seringkali dapat memberikan kesimpulan yang menyesatkan jika diterapkan pada dataset yang memiliki *data imbalance* yang ekstrem. Model bisa saja mendapatkan nilai *accuracy* yang tinggi hanya karena terus-menerus menebak kelas mayoritas secara benar, namun gagal total dalam mendeteksi kelas minoritas. Oleh karena itu, *Classification Report* diimplementasikan sebagai metode evaluasi lanjutan untuk membedah performa model secara spesifik pada tiap kategori bencana.

Classification Report memberikan rincian kinerja model melalui empat indikator evaluasi utama pada masing-masing kelas:

- **Precision:** Mengukur tingkat ketepatan model dalam memprediksi. Indikator ini menunjukkan dari seluruh gambar yang diprediksi oleh model sebagai suatu jenis bencana tertentu, berapa persen yang tebakannya benar.
- **Recall:** Mengukur tingkat sensitivitas model. Indikator ini menunjukkan dari seluruh jumlah gambar aktual pada suatu jenis bencana, berapa persen yang berhasil ditebak dengan benar oleh model. Nilai *recall* sangat krusial dalam menguji keberhasilan model mengenali kelas minoritas (seperti Earthquake).
- **F1-Score:** Merupakan nilai rata-rata harmonik dari *precision* dan *recall*. Metrik ini adalah indikator paling ideal untuk mengukur performa model pada kasus *data imbalance*, karena memberikan keseimbangan antara tingkat ketepatan dan tingkat kepekaan prediksi model.
- **Support:** Merepresentasikan jumlah kemunculan aktual dari setiap label kelas yang ada di dalam kumpulan data uji.

Melalui hasil *Classification Report* ini, evaluasi terhadap keandalan metode *class weight* maupun *oversampling* dapat dilakukan secara terukur. Analisis tidak hanya melihat model mana yang berkinerja baik secara umum, tetapi juga secara spesifik mengevaluasi model mana yang paling berhasil menyelamatkan performa deteksi pada kelas minoritas tanpa harus merusak performa prediksi pada kelas mayoritas.

8.3 Confusion Matrix

Sebagai pelengkap dari metrik evaluasi sebelumnya, *Confusion Matrix* digunakan untuk memvisualisasikan performa model klasifikasi secara lebih transparan dan mendetail. *Confusion Matrix* memetakan secara langsung jumlah absolut tebakan yang benar maupun yang salah ke dalam sebuah *grid* matriks dua dimensi.

Sumbu Y (vertikal) pada matriks ini merepresentasikan label kelas yang sebenarnya (*True Label*), sedangkan sumbu X (horizontal) merepresentasikan label prediksi yang dihasilkan oleh model (*Predicted Label*). Kinerja model yang optimal akan ditunjukkan oleh penumpukan angka yang tinggi pada garis diagonal utama matriks (dari ujung kiri atas ke ujung kanan bawah), yang menandakan *True Positives* atau kondisi di mana tebakan model sesuai dengan kenyataan. Sebaliknya, angka yang

berada di luar garis diagonal tersebut merepresentasikan kesalahan prediksi (*False Positives* dan *False Negatives*).

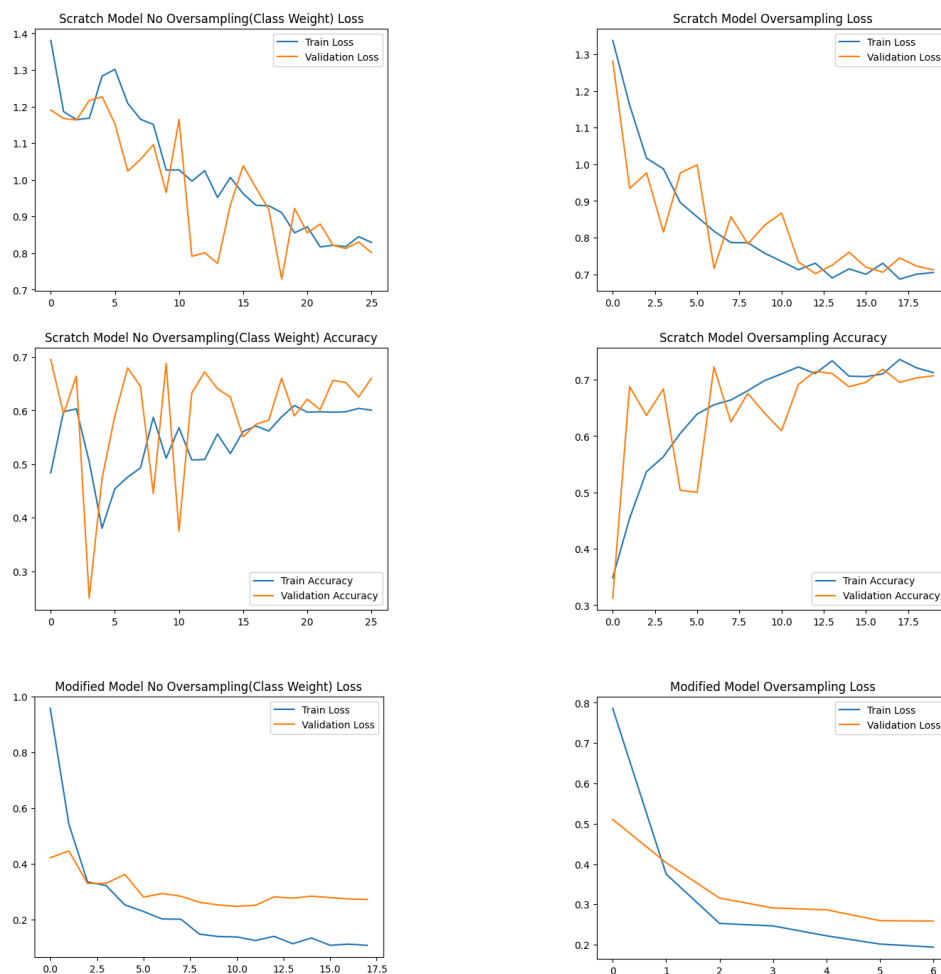
Dalam konteks penelitian ini yang memiliki tantangan *data imbalance* yang signifikan, *Confusion Matrix* menjadi instrumen analitis yang sangat krusial untuk membedah pola kesalahan model.

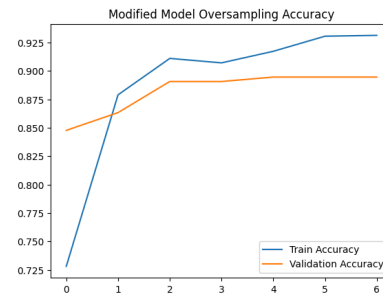
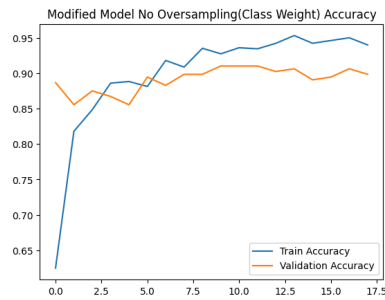
III. Hasil dan Analisis

Penelitian ini menggunakan total empat pendekatan pemodelan untuk diuji dan dianalisis. Keempat pendekatan tersebut terdiri dari dua model yang dibangun dari awal (*from scratch*) dan dua model *transfer learning* menggunakan arsitektur EfficientNetB1. Pada masing-masing arsitektur, kami menerapkan dua skenario penanganan ketidakseimbangan data, yakni metode *class weight* (tanpa *oversampling*) dan metode *oversampling*.

1. Analisis Kurva Pelatihan

Berikut adalah rincian *learning curve* untuk setiap model:





Jika kita mengamati grafik kurva *learning curve* pada model Scratch, baik yang menggunakan metode *class weight* maupun *oversampling*, keduanya menunjukkan pergerakan yang sangat tidak stabil. Ketidakstabilan ini menandakan bahwa kedua model masih kesulitan untuk menemukan pola yang tepat dari data yang diberikan. Secara lebih spesifik, pada model Scratch *Oversampling*, tingkat *train accuracy* terlihat terus meningkat, namun *validation accuracy* berfluktuasi naik turun dengan tajam. Kondisi ini adalah indikasi jelas terjadinya *overfitting*. Hal ini sangat wajar terjadi mengingat adanya ketimpangan data yang ekstrem. Kelas Earthquake hanya memiliki 36 data, sedangkan Water Disaster memiliki 1035 data. Akibatnya, pada metode *oversampling*, model cenderung hanya menghafal data Earthquake yang disalin dan diulang-ulang agar jumlahnya setara dengan kelas terbanyak.

Di sisi lain, grafik pada model *transfer learning* (menggunakan arsitektur EfficientNetB1) menunjukkan hasil yang berbeda. Baik pada metode *class weight* maupun *oversampling*, metrik validasi mulai memasuki fase pelandaian (*plateau*). Fase ini mengindikasikan bahwa model telah mencapai batas maksimal kemampuannya dalam mengekstrak fitur dari data yang tersedia. Sementara itu, nilai *train loss* dan *accuracy* pada data pelatihan terus membaik seiring bertambahnya *epoch*. Hal ini menciptakan jarak (*gap*) yang semakin melebar antara kurva pelatihan dan kurva validasi. Pelebaran jarak tersebut merupakan indikator kuat terjadinya *overfitting* ringan, yang kemungkinan besar dipicu oleh kondisi *data imbalance*.

Jika hanya melihat sekilas pada grafik, model *transfer learning* dengan *oversampling* memang tampak lebih stabil dibandingkan metode *class weight*. Namun, mengingat distribusi data awal yang sangat timpang, kestabilan pada *Oversampling* ini sangat berpotensi menjadi sebuah ilusi performa. Model tersebut terlihat bagus semata-mata karena ia diuji menggunakan data duplikat yang sudah ia hafal luar dalam selama proses pelatihan. Oleh karena itu, dapat disimpulkan bahwa meskipun grafik *transfer learning Oversampling* terlihat lebih mulus, model **Transfer Learning dengan metode Class Weight** tetap menjadi pilihan yang paling *robust* dan dapat diandalkan (*reliable*) untuk kasus ketimpangan data yang ekstrem ini.

2. Analisis Pemilihan Metrik Evaluasi

Dalam konteks tanggap darurat bencana alam, pemilihan metrik evaluasi model klasifikasi gambar sangatlah krusial. Kami tidak menggunakan metrik *Accuracy* karena

hasilnya sangat menipu, terutama pada kondisi persebaran data yang sangat tidak seimbang (*imbalanced data*). Selain itu, kami menghindari fokus tunggal pada *Recall* atau *Precision*. Jika berfokus pada *Recall*, model akan terlalu sensitif dan banyak mengklasifikasikan foto kejadian biasa sebagai bencana. Hal ini akan membuang waktu dan sumber daya tim penyelamat yang terbatas untuk mengecek lokasi yang salah. Sebaliknya, jika berfokus pada *Precision*, model menjadi terlalu berhati-hati. Foto bencana sungguhan yang visualnya kurang jelas (abu-abu) berpotensi diabaikan, sehingga bantuan untuk korban akan terlambat. Oleh karena itu, kami memilih *F1-Score* untuk mendapatkan keseimbangan yang optimal.

Lebih spesifik lagi, dalam penggunaan *F1-Score* ini, kami berfokus pada nilai **Macro Average** dan menghindari **Weighted Average**. Secara cara kerja, *Weighted Average* menghitung rata-rata skor dengan memberikan bobot lebih besar pada kelas yang jumlah datanya paling banyak. Jika kami menggunakan metrik ini, skor evaluasi model akan didominasi oleh kelas Water Disaster yang datanya melimpah, sehingga kelemahan fatal model dalam mendeteksi kelas minoritas seperti Earthquake akan tertutupi. Sebaliknya, *Macro Average* menghitung nilai rata-rata dengan menjumlahkan skor dari setiap kelas lalu membaginya sama rata, tanpa memperdulikan jumlah data di dalamnya. Pendekatan *Macro Average* ini memastikan setiap jenis bencana dianggap sama pentingnya. Hal ini menuntut model untuk benar-benar andal dalam mengenali semua jenis bencana secara merata, bukan hanya terlihat bagus karena didongkrak oleh prediksi pada kelas mayoritas.

3. Evaluasi Classification Report

Berikut adalah rincian *classification report* untuk setiap model:

Scratch Model	No Oversampling (Class Weight)			
	precision	recall	f1-score	support
Earthquake	0.15	0.57	0.24	7
Land_Slide	0.44	0.39	0.41	57
Urban_Fire	0.82	0.63	0.71	65
Water_Disaster	0.76	0.77	0.77	165
accuracy			0.66	294
macro avg	0.54	0.59	0.53	294
weighted avg	0.70	0.66	0.67	294

Scratch Model	Oversampling			
	precision	recall	f1-score	support
Earthquake	0.12	0.29	0.17	7
Land_Slide	0.42	0.54	0.48	57
Urban_Fire	0.89	0.62	0.73	65
Water_Disaster	0.81	0.78	0.79	165
accuracy			0.68	294
macro avg	0.56	0.56	0.54	294
weighted avg	0.73	0.68	0.70	294

Transfer Model	No Oversampling (Class Weight, EfficientNetB1)			
	precision	recall	f1-score	support
Earthquake	0.50	0.57	0.53	7
Land_Slide	0.67	0.81	0.73	57
Urban_Fire	1.00	0.78	0.88	65
Water_Disaster	0.93	0.93	0.93	165
accuracy			0.87	294
macro avg	0.77	0.77	0.77	294
weighted avg	0.88	0.87	0.87	294

Transfer Model	Oversampling (EfficientNetB1)			
	precision	recall	f1-score	support
Earthquake	0.28	0.71	0.40	7
Land_Slide	0.54	0.72	0.62	57
Urban_Fire	0.98	0.82	0.89	65
Water_Disaster	0.95	0.84	0.89	165
accuracy			0.81	294
macro avg	0.69	0.77	0.70	294
weighted avg	0.86	0.81	0.82	294

Berdasarkan hasil evaluasi Classification Report, terlihat adanya peningkatan performa yang signifikan saat menggunakan pendekatan *transfer learning* dibandingkan dengan model *scratch*. Evaluasi ini berfokus pada metrik *F1-Score Macro*, yang merupakan metrik evaluasi paling objektif untuk menilai performa model pada dataset dengan ketimpangan distribusi kelas yang tinggi. Nilai *F1-Score Macro* menunjukkan peningkatan dari kisaran 0,53 dan 0,54 pada model *scratch* menjadi 0,70 hingga 0,77 pada model *transfer learning*.

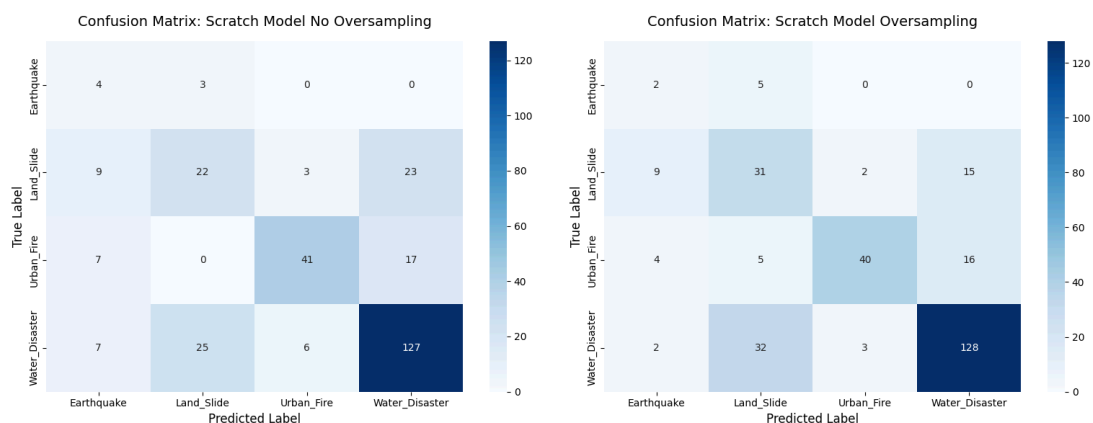
Di antara kedua variasi *transfer learning* tersebut, metode *Class Weight* mencatatkan performa keseluruhan yang terbaik dan paling stabil dengan nilai *F1-Score Macro* mencapai 0,77.

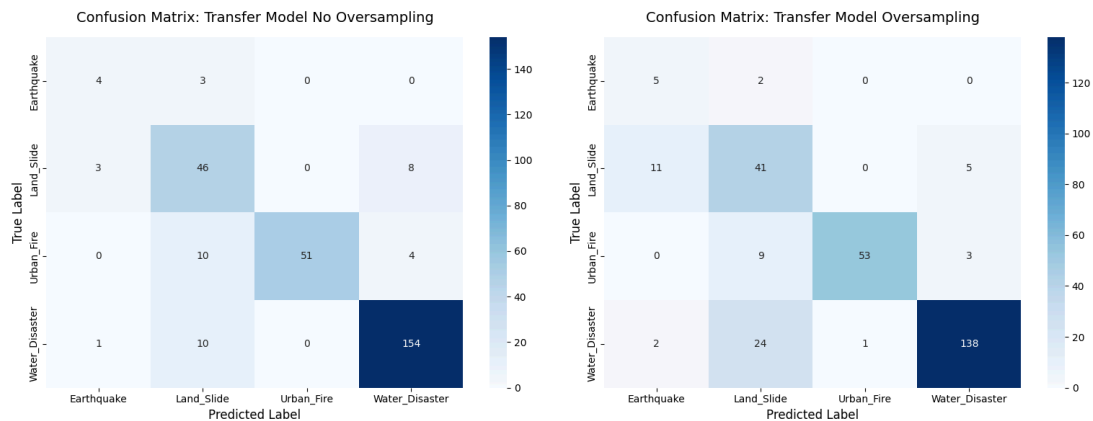
Evaluasi lebih mendalam pada masing masing kelas, khususnya pada kelas minoritas Earthquake, memperjelas kelemahan dari penggunaan metode *oversampling*. Model *transfer learning Oversampling* memang mencatatkan nilai *recall* yang tinggi di angka 0,71, yang berarti model memiliki kepekaan yang baik untuk menemukan gambar gempa bumi. Namun, nilai *precision* pada model tersebut turun drastis menjadi 0,28. Hal ini menandakan tingginya prediksi *False Positive*, di mana model sering keliru menebak gambar bencana lain sebagai gempa bumi karena terlalu terbiasa dengan data duplikat selama pelatihan. Sebaliknya, *transfer learning Class Weight* memberikan proporsi yang lebih masuk akal antara ketepatan *precision* 0,50 dan *recall* 0,57. Keseimbangan ini menghasilkan nilai akhir F1-Score sebesar 0,53 yang jauh lebih dapat diandalkan.

Keseluruhan hasil dari metrik evaluasi ini sejalan dengan indikasi yang terlihat pada grafik *learning curve* sebelumnya. Model *transfer learning Oversampling* terbukti mengalami kendala generalisasi saat diuji karena model terlalu fokus menghafal data duplikat pada kelas minoritas. Berdasarkan rincian evaluasi tersebut, dapat ditarik kesimpulan bahwa *transfer learning Class Weight* adalah pendekatan yang paling stabil, seimbang, dan merupakan solusi paling tepat untuk menangani masalah ketidakseimbangan data yang ekstrem pada penelitian ini.

4. Analisis Confusion Matrix

Berikut adalah rincian Confusion Matrix untuk setiap model:





Berdasarkan analisis Confusion Matrix, model Transfer Learning (EfficientNetB1) dengan metode *class weight* menunjukkan kinerja yang paling optimal karena memiliki tingkat kesalahan tebak (*misclassification*) yang paling rendah secara keseluruhan. Distribusi tebakan benar dan salah pada matriks ini memberikan gambaran yang sangat jelas mengenai kemampuan model dalam mengenali detail setiap kelas bencana. Berikut adalah rincian performa model pada masing-masing kelas:

- Earthquake: 4 gambar diprediksi benar (*True Positive*), 3 gambar diprediksi salah (*False Negative*). Model berhasil mengenali lebih dari separuh data uji pada kelas ini.
- Landslide: 46 gambar diprediksi benar (*True Positive*), 11 gambar diprediksi salah (*False Negative*).
- Urban Fire: 51 gambar diprediksi benar (*True Positive*), 14 gambar diprediksi salah (*False Negative*).
- Water Disaster: 154 gambar diprediksi benar (*True Positive*), 11 gambar diprediksi salah (*False Negative*).

Berdasarkan pola sebaran kesalahan pada *confusion matrix* tersebut, dapat ditarik kesimpulan bahwa model memiliki keandalan yang sangat baik dalam mendeteksi kejadian Urban Fire. Secara visual, kelas ini memiliki fitur warna dominan yang sangat mencolok, seperti warna merah atau oranye dari api. Fitur kuat inilah yang membuatnya jauh lebih mudah diekstrak dan jarang tertukar dengan kelas lainnya. Sebaliknya, kebingungan model lebih sering terjadi di antara kelas Earthquake, Landslide, dan Water Disaster. Hal ini sangat logis mengingat elemen visual pada ketiga bencana tersebut cenderung memiliki palet warna yang saling tumpang tindih, seperti dominasi warna kecoklatan dari tanah, puing reruntuhan bangunan, atau air yang keruh bercampur lumpur. Kemiripan fitur visual inilah yang menjadi alasan utama mengapa model masih sedikit kesulitan untuk membedakan batas pola yang kuat di antara ketiga kelas tersebut.

IV. Kesimpulan

Berdasarkan keseluruhan rangkaian penelitian yang telah dilakukan, dapat disimpulkan bahwa tantangan utama dalam pengembangan model klasifikasi bencana alam ini bersumber dari kondisi dataset yang digunakan. Dataset ini memiliki tingkat ketidakseimbangan kelas (*data imbalance*) yang sangat ekstrem, khususnya pada kelas Earthquake. Meskipun upaya penanganan telah diterapkan melalui metode pembobotan kelas (*class weight*) maupun *oversampling*, jurang ketimpangan data yang terlalu besar tetap menjadi kendala yang nyata. Akibatnya, model mengalami kesulitan untuk belajar secara optimal dalam mengekstrak dan mengenali pola dari kelas minoritas tersebut.

Hasil evaluasi pengujian menunjukkan bahwa model terbaik yang dihasilkan dalam penelitian ini adalah model Transfer Learning (EfficientNetB1) yang menggunakan metode *class weight*. Namun demikian, proses pembelajaran pada model ini terpantau mengalami *plateau*. Kondisi ini mengindikasikan bahwa model telah berhasil menyerap seluruh fitur relevan dari data yang tersedia, tetapi kesulitan untuk menemukan variasi pola baru. Hal ini sangat wajar terjadi karena terbatasnya keragaman pola visual di dalam dataset itu sendiri.

Fenomena pelandaian (*plateau*) ini juga sangat erat kaitannya dengan cara kerja dasar dari pendekatan *transfer learning*. Arsitektur EfficientNetB1 telah melalui tahap pra-pelatihan (*pre-trained*) menggunakan ImageNet, sehingga model ini sudah memiliki pondasi kemampuan yang sangat matang dalam mendeteksi fitur visual dasar seperti garis tepi, transisi warna, bayangan, dan tekstur. Oleh karena itu, tugas utama model pada penelitian ini tidak lagi mempelajari elemen visual dari nol, melainkan langsung mengaitkan elemen dasar tersebut untuk memahami konsep spesifik dari setiap kelas bencana alam, inilah yang membuat model mampu mengenali fitur dataset dengan sangat cepat, yang pada akhirnya mempercepat model mencapai batas maksimal pembelajarannya dan masuk ke fase *plateau*.

Secara keseluruhan, model Transfer Learning (EfficientNetB1) dengan metode *class weight* ini dinilai sudah cukup layak untuk diimplementasikan (*deploy*). Meskipun demikian, terdapat beberapa langkah perbaikan yang sangat direkomendasikan untuk pengembangan model di masa depan, antara lain:

1. **Pengumpulan Data yang Seimbang:** Model perlu dilatih ulang (*retrain*) menggunakan komposisi dataset yang lebih berimbang. Langkah ini sangat krusial untuk menekan bias prediksi antar kelas yang saat ini masih terjadi akibat dominasi kelas mayoritas.
2. **Penerapan GAN untuk Variasi Data:** Selain menggunakan teknik augmentasi gambar konvensional, teknologi *Generative Adversarial Networks* (GAN) dapat dimanfaatkan untuk menghasilkan gambar sintesis yang realistis. Pendekatan ini

diharapkan dapat memberikan variasi pola data yang lebih kaya dan relevan, khususnya untuk memperbanyak sampel pada kelas minoritas.

V. Refrensi

UNDRR. (2015). *Sendai Framework for Disaster Risk Reduction 2015-2030* (Bagian IV, Paragraf 24 Poin k). Geneva: United Nations Office for Disaster Risk Reduction.

VI. Lampiran

Google Collab: [Link](#)

Video Penjelasan: [Link](#)